

Министерство науки и высшего образования РФ
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЁТ

по лабораторной работе №2

по дисциплине «Логика и основы алгоритмизации в инженерных задачах»

на тему «Оценка времени выполнения программ»

Выполнили ст. гр. 25BVB1:

Димов Н.А.

Бардычев А.А.

Приняли:

к.т.н., доцент Юрова О. В.

ст. преподаватель Деев М. В.

Пенза 2026

1. Цель работы

Изучение практических методов измерения времени выполнения фрагментов кода и программ в целом с использованием функций библиотеки `<time.h>` (`<ctime>`), расчет и оценка асимптотической сложности алгоритмов (О-символика), а также экспериментальный анализ производительности алгоритма перемножения квадратных матриц и алгоритмов сортировки (сортировка Шелла, быстрая сортировка `qs` Хоара и системная функция `qsort`).

2. Лабораторное задание

Задание 1

- Вычислить порядок временной сложности алгоритма умножения квадратных матриц (О-символику).
- Оценить время выполнения программы и блока перемножения матриц с помощью функций библиотеки `<time.h>` для квадратных матриц размерами $N \in \{100, 200, 400, 1000, 2000, 4000, 10000\}$.
- Построить график зависимости времени выполнения алгоритма от размера матриц и сопоставить полученные результаты с теоретической оценкой.

Задание 2

Собрать метрики времени работы алгоритма сортировки Шелла (`shell`), быстрой сортировки (`qs`) и системной функции `qsort()` на различных наборах данных ($N = 10000$) при 4 типах начальной упорядоченности:

- Случайный набор чисел;
- Возрастающая последовательность элементов;
- Убывающая последовательность элементов;
- Пилообразная последовательность (первая половина — по возрастанию, вторая — по убыванию).

3. Теоретический анализ алгоритмов

3.1. Умножение матриц

Рассматриваемый алгоритм выполняет перемножение двух квадратных матриц A и B размером $N \times N$. Результат записывается в матрицу C . Вычисление каждого элемента $c[i][j]$ сводится к скалярному произведению i -й строки матрицы A и j -го столбца матрицы B :

$$c[i][j] = a[i][0] \cdot b[0][j] + a[i][1] \cdot b[1][j] + \dots + a[i][N-1] \cdot b[N-1][j].$$

Алгоритм содержит три вложенных цикла:

- внешний цикл по строкам i — N итераций;
- цикл по столбцам j — N итераций для каждой строки;
- внутренний цикл по индексу r — N итераций для каждой пары (i, j) .

Следовательно, общее число операций умножения и сложения имеет порядок:

$$N \cdot N \cdot N = N^3.$$

Таким образом, теоретическая временная сложность алгоритма составляет:

$$O(N^3).$$

Из этой оценки следует, что при увеличении размера матрицы в 2 раза время выполнения должно возрасть примерно в $2^3 = 8$ раз.

3.2. Алгоритмы сортировки

- **Сортировка Шелла (shell).** Представляет собой модификацию сортировки вставками, в которой используется фиксированная последовательность зазоров: $\{9, 5, 3, 2, 1\}$. На почти упорядоченных массивах алгоритм выполняет минимальное число перестановок и работает достаточно быстро, вплоть до $O(N)$. Однако на обратно упорядоченных данных из-за ограниченного набора шагов сложность может ухудшаться до $O(N^2)$.
- **Быстрая сортировка Хоара (qs).** Является рекурсивным алгоритмом. В качестве опорного элемента выбирается центральный элемент подмассива: $\text{items}[(\text{left} + \text{right}) / 2]$. В среднем случае сложность составляет $O(N \log N)$. Выбор центрального элемента обеспечивает достаточно равномерное разбиение и высокую скорость работы как на возрастающих, так и на убывающих массивах.
- **Стандартная функция `qsort()`.** Это библиотечная реализация сортировки из `<stdlib.h>`. Она использует оптимизированный вариант быстрой сортировки с защитой от худших случаев и обеспечивает устойчивую сложность порядка $O(N \log N)$.

4. Исходный код программы

В соответствии с заданием полный листинг программы в отчёт не включается. Исходный код оформлен как консольное приложение в проекте Visual Studio (файл alg2lr.cpp). Программа написана на языке C и содержит:

- функцию `shell()` — реализацию сортировки Шелла;
- функцию `qs()` — реализацию быстрой сортировки Хоара;
- функцию `task1()` — перемножение квадратных матриц с динамическим выделением памяти для размеров до $N = 10000$;
- функцию `task2()` — измерение времени работы алгоритмов сортировки.

5. Результаты работы программы

5.1. Задание 1: Зависимость времени умножения матриц от размера N

Экспериментальные замеры времени умножения квадратных матриц приведены в таблице 1. Замер для матрицы $N = 10000$ был успешно выполнен в полном объеме.

Таблица 1 – Зависимость времени умножения матриц от размера N

Размер матрицы (N×N)	Число операций (N^3)	Время умножения t1 (с)	Общее время работы t2 (с)
100	1×10^6	0.002	0.002
200	8×10^6	0.014	0.014
400	64×10^6	0.115	0.115
1000	1×10^9	2.064	2.064
2000	8×10^9	20.665	20.665
4000	6.4×10^{10}	231.061	231.061
10000	1×10^{12}	4507.915	4507.915

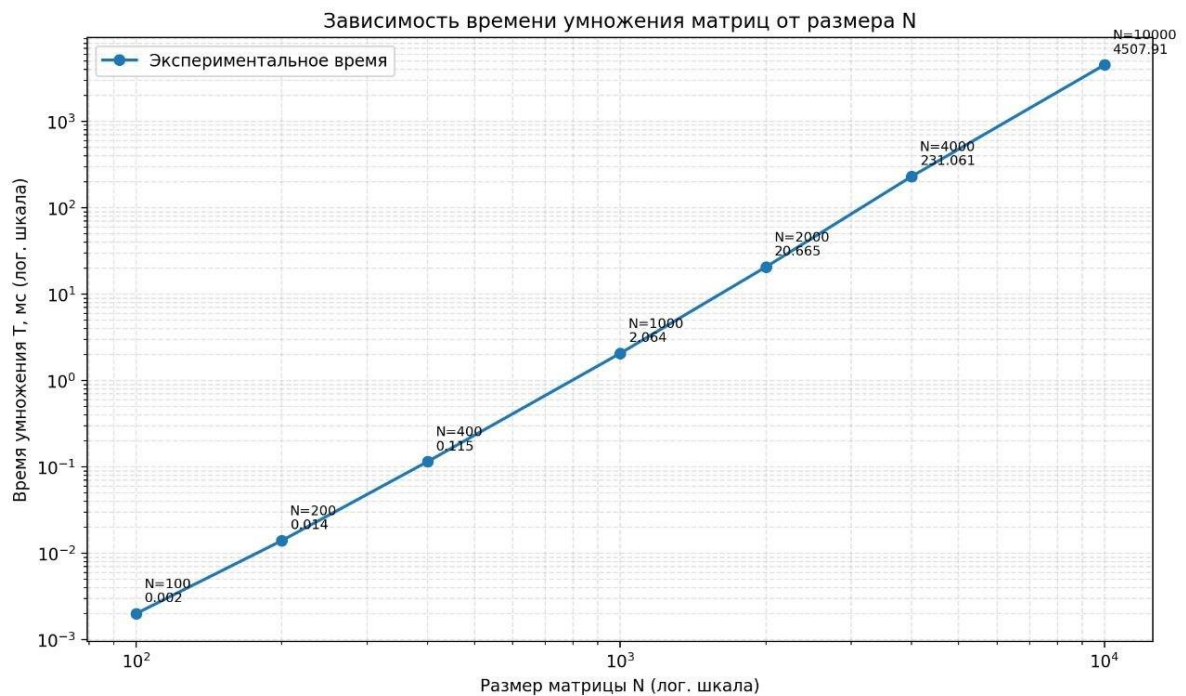


Рисунок 1 – График зависимости времени умножения матриц T от размера N

Анализ результатов Задания 1:

1. На интервале размеров N от 100 до 2000 экспериментальные замеры в целом согласуются с теоретической кубической сложностью $O(N^3)$. При удвоении размера матрицы время вычислений возрастает примерно в 8–10 раз: например, переходы $0,002 \rightarrow 0,014 \rightarrow 0,115$ с дают увеличение в 7 и 8,2 раза соответственно, а переход $1000 \rightarrow 2000$ ($2,064 \rightarrow 20,665$ с) показывает рост примерно в 10 раз. Это подтверждает теоретический закон $2^3 = 8$ с учетом накладных расходов.
2. При дальнейшем увеличении размерности до $N = 4000$ и $N = 10000$ наблюдается более выраженное замедление относительно теоретической оценки. Время выполнения для $N = 10000$ составило 4307,915с (около 71,8 мин), что превышает строго кубический рост. Это обусловлено эффектами иерархии памяти: при больших N матрицы перестают помещаться в быстрый кэш процессора. Поскольку во внутреннем цикле обход второй матрицы идет по столбцам, это вызывает дополнительные промахи кэша (cache misses) и увеличивает время доступа к оперативной памяти.

5.2. Задание 2: Сравнительный анализ алгоритмов сортировки

В таблице 2 приведены результаты замеров времени работы алгоритмов сортировки для массива размером $N = 10000$ элементов при различных типах начальной упорядоченности.

Таблица 2 – Время выполнения алгоритмов сортировки для $N = 10000$ (в секундах)

Тип входной последовательности	Сортировка Шелла (shell)	Быстрая сортировка (qs)	Системная qsort()
1. Случайный массив	0.004 с	0.001 с	0.001 с
2. Возрастающий массив	0.0000 с	0.001 с	0.001 с
3. Убывающий массив	0.009 с	0.0000 с	0.001 с
4. Пилообразный массив (1/2 возр., 1/2 убыв.)	0.0000 с	0.0000 с	0.002 с

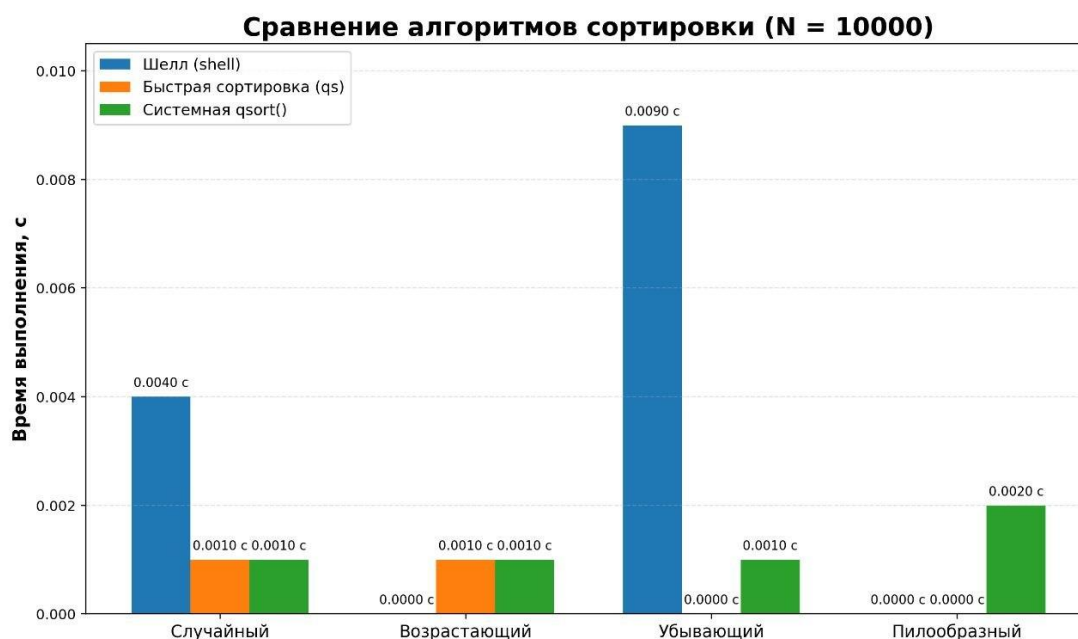


Рисунок 2 – Сравнение времени работы алгоритмов сортировки ($N = 10000$)

Анализ результатов Задания 2:

1. Быстрая сортировка Хоара (qs) продемонстрировала максимальную скорость на всех типах данных. На убывающем и пилообразном массивах время работы оказалось настолько малым, что не было зафиксировано таймером (0.0000 с), а на случайном и возрастающем составило порядка 0.001 с. Это подтверждает эффективность выбора центрального элемента в качестве опорного: разбиение остается сбалансированным даже на предварительно упорядоченных данных.

2. Системная функция `qsort()` показала стабильный результат на уровне 0.001–0.002 с. Она незначительно уступает `qs` на случайных и частично упорядоченных данных, что связано с накладными расходами на вызов функции-компаратора через указатель. Тем не менее, `qsort()` демонстрирует высокую устойчивость к типу входной последовательности.
3. Сортировка Шелла (`shell`) оказалась наиболее чувствительной к исходному порядку элементов. На возрастающем и пилообразном массивах она сработала быстрее разрешения таймера (0.0000 с). Однако на случайном массиве время возросло до 0.004 с, а на убывающем достигло 0.009 с. Это объясняется тем, что при фиксированном наборе шагов `gar` обратно упорядоченный массив требует максимального числа перестановок.

6. Выводы

- Освоены практические методы измерения времени выполнения программ на языке C с использованием функций библиотеки `<time.h>` (`clock()`, `CLOCKS_PER_SEC`).
- Экспериментально подтверждена теоретическая кубическая сложность $O(N^3)$ алгоритма перемножения матриц. Выполнен полный замер для матрицы $N = 10000$, время выполнения которого составило 4307,915с (**примерно 71,8 минуты**). Это подтвердило существенное влияние промахов кэш-памяти и пропускной способности подсистемы памяти на производительность при обработке больших объёмов данных.
- Проведено сравнительное тестирование трёх алгоритмов сортировки (Шелла, `qs` Хоара, `qsort`) на 4 типах начальной упорядоченности данных ($N = 10000$).
- **Быстрая сортировка `qs`** подтвердила свою высокую эффективность: на ряде наборов данных время работы оказалось ниже разрешающей способности таймера (`clock()`), что говорит о её практической мгновенности для данного объёма данных.
- **Стандартная функция `qsort()`** показала стабильное время работы на уровне 1–2 мс независимо от типа входных данных, немного уступая `qs` из-за накладных расходов на компаратор, но оставаясь надёжным инструментом.
- **Сортировка Шелла** подтвердила зависимость от структуры данных: она крайне быстра на почти отсортированных массивах, но существенно замедляется на случайных (0.004 с) и обратно упорядоченных (0.009 с) последовательностях.